05-02 Accounts DB 구현 분석

개요

Solana의 Accounts DB는 블록체인의 모든 계정 데이터를 효율적으로 저장, 검색, 관리하는 핵심 컴포넌트입니다. 고성능 트랜잭션 처리를 위해 메모리 매핑된 파일, 인덱싱, 캐싱 등 다양한 최적화 기법을 사용합니다.

AccountsDb 구조체 분석

핵심 구조체 정의

```
#[derive(Debug)]
pub struct AccountsDb {
/// Keeps tracks of index into AppendVec on a per slot basis
pub accounts_index: AccountInfoAccountsIndex,
/// Some(offset) iff we want to squash old append vecs together into 'ancient ap
pub ancient_append_vec_offset: Option<i64>,
/// true iff we want to skip the initial hash calculation on startup
pub skip_initial_hash_calc: bool,
/// Account storage management
pub storage: AccountStorage,
/// In-memory cache for frequently accessed accounts
pub accounts_cache: AccountsCache,
```

```
/// Write cache limit in bytes
write_cache_limit_bytes: Option<u64>,
/// Background hasher for cache entries
sender_bg_hasher: Option<Sender<CachedAccount>>,
/// Read-only accounts cache for optimization
read_only_accounts_cache: ReadOnlyAccountsCache,
/// Atomic counter for append vec IDs
pub  next_id: AtomicAppendVecId,
/// Set of shrinkable stores organized by slot
pub  shrink_candidate_slots: Mutex<ShrinkCandidates>,
/// Global write version counter
pub  write_version: AtomicU64,
/// Set of storage paths to pick from
pub  paths: Vec<PathBuf>,
/// Thread pool for parallel operations
pub  thread_pool: ThreadPool,
pub  thread_pool_clean: ThreadPool,
/// Various hash caches and statistics
bank_hash_stats: Mutex<HashMap<Slot, BankHashStats>>,
accounts_delta_hashes: Mutex<HashMap<Slot, AccountsDeltaHash>>,
accounts_hashes: Mutex<HashMap<Slot, (AccountsHash, u64)>>,
// ... 기타 필드들
```

```
}
```

주요 타입 정의

```
type AccountInfoAccountsIndex = AccountsIndex<AccountInfo, AccountInfo>;

type ShrinkCandidates = IntSet<Slot>;

type PubkeysRemovedFromAccountsIndex = HashSet<Pubkey>;
```

계정 저장 메커니즘

1. 저장 구조

Accounts DB는 다음과 같은 계층적 저장 구조를 사용합니다:

- **AppendVec**: 메모리 매핑된 파일로 계정 데이터 저장
- **AccountsIndex**: 계정 위치 정보 인덱스
- **AccountsCache**: 메모리 내 캐시
- **ReadOnlyAccountsCache**: 읽기 전용 캐시

2. 계정 저장 프로세스

```
pub fn store_cached<'a, T: ReadableAccount + Sync + ZeroLamport + 'a>(

&self,

accounts: impl StorableAccounts<'a, T>,

transactions: Option<&'a [Option<&'a SanitizedTransaction>]>,

) {
```

```
self.store(
accounts,
&StoreTo::Cache,
transactions,
StoreReclaims::Default,
UpdateIndexThreadSelection::PoolWithThreshold,
);
}
```

저장 과정:

1. **캐시 우선 저장**: 새로운 계정은 먼저 메모리 캐시에 저장
2. **인덱스 업데이트**: AccountsIndex에 계정 위치 정보 업데이트
3. **백그라운드 플러시**: 캐시가 가득 차면 디스크로 플러시
4. **버전 관리**: 각 업데이트마다 write_version 증가

3. StoreTo 열거형

```
enum StoreTo<'a> {
/// write to cache
Cache,
/// write to storage
Storage(&'a Arc<AccountStorageEntry>),
}
```

계정 검색 메커니즘

1. 계정 로드 프로세스

```
pub fn load(
&self,
ancestors: &Ancestors,
pubkey: &Pubkey,
load_hint: LoadHint,
) -> Option<(AccountSharedData, Slot)> {
self.do_load(ancestors, pubkey, None, load_hint, LoadZeroLamports::None)
}
```

로드 과정:

1. **캐시 확인**: 먼저 AccountsCache에서 계정 검색
2. **읽기 전용 캐시 확인**: ReadOnlyAccountsCache 검색
3. **인덱스 조회**: AccountsIndex에서 계정 위치 정보 조회
4. **스토리지 접근**: AppendVec에서 실제 계정 데이터 로드

2. LoadedAccountAccessor

```
enum LoadedAccountAccessor<'a> {
// AccountStorageEntry
Stored(Option<(Arc<AccountStorageEntry>, usize)>),
// None value in Cached variant means the cache was flushed
```

```
Cached(Option<Cow<'a, CachedAccount>>),
}
```

인덱싱 시스템

1. AccountsIndex 구조

AccountsIndex는 계정의 pubkey를 키로 하여 계정의 위치 정보를 저장합니다:

```
pub accounts_index: AccountInfoAccountsIndex,
```

2. 인덱스 업데이트

```
// 인덱스에서 계정 정보 조회
self.accounts_index.get_and_then(entry.key(), |index_entry| {
if let Some(index_entry) = index_entry {
// 참조 카운트 및 슬롯 정보 처리
match (index_entry.ref_count() as usize).cmp(&entry.value().len()) {
// 참조 카운트 검증 로직
}
}
});
```

3. 스캔 기능

```
self.accounts_index.scan(
pubkeys.iter(),
|pubkey, slots_refs, _entry| {
// 각 계정에 대한 처리 로직
if  let Some((slot_list, ref_count)) = slots_refs {
let  index_in_slot_list = self.accounts_index.latest_slot(
None,
slot_list,
max_clean_root_inclusive,
);
// 슬롯 리스트 처리
}
AccountsIndexScanResult::KeepInMemory
},
);
```

캐싱 메커니즘

1. 다층 캐시 구조

Accounts DB는 여러 레벨의 캐시를 사용합니다:

AccountsCache (쓰기 캐시)

```
pub accounts_cache: AccountsCache,
write_cache_limit_bytes: Option<u64>,
```

- 새로 생성되거나 수정된 계정을 임시 저장
- 기본 제한: 15GB (`WRITE_CACHE_LIMIT_BYTES_DEFAULT`)
- 백그라운드에서 디스크로 플러시

ReadOnlyAccountsCache (읽기 캐시)

```
read_only_accounts_cache: ReadOnlyAccountsCache,
```

- 자주 읽히는 계정을 메모리에 캐시
- 읽기 성능 최적화

2. 캐시 플러시 메커니즘

```
pub fn load_account_into_read_cache(&self, ancestors: &Ancestors, pubkey: &Pub
self.do_load_with_populate_read_cache(
ancestors,
pubkey,
None,
LoadHint::Unspecified,
true,
);
```

```
}
```

정리 및 압축 (Cleaning and Shrinking)

1. 계정 정리 프로세스

```
fn clean_accounts_older_than_root(
&self,
purges: impl Iterator<Item = (Pubkey, (SlotList<AccountInfo>, RefCount))>,
max_clean_root_inclusive: Option<Slot>,
ancient_account_cleans: &AtomicU64,
epoch_schedule: &EpochSchedule,
) -> (ReclaimResult, PubkeysRemovedFromAccountsIndex) {
// 오래된 계정 버전 정리
let removed_from_index = self.accounts_index.clean_rooted_entries(
pubkey,
&mut reclaims,
max_clean_root_inclusive,
);
// 정리된 계정들 반환
}
```

2. 스토리지 압축

```
pub shrink_candidate_slots: Mutex<ShrinkCandidates>,
```

- 사용률이 낮은 AppendVec 파일들을 압축
- 디스크 공간 효율성 향상
- 백그라운드에서 수행

해시 계산 및 검증

1. 계정 해시 관리

```
accounts_hashes: Mutex<HashMap<Slot, (AccountsHash, /*capitalization*/ u64)>>,
accounts_delta_hashes: Mutex<HashMap<Slot, AccountsDeltaHash>>,
incremental_accounts_hashes: Mutex<HashMap<Slot, (IncrementalAccountsHash, /*cap
```

2. 해시 계산 메서드

```
pub fn get_accounts_hash(&self, slot: Slot) -> Option<(AccountsHash, /*capital
self.accounts_hashes.lock().unwrap().get(&slot).cloned()
}

pub fn get_accounts_delta_hash(&self, slot: Slot) -> Option<AccountsDeltaHash>
self.accounts_delta_hashes
.lock()
.unwrap()
```

```
.get(&slot)

.cloned()

}
```

성능 최적화

1. 병렬 처리

```
pub  thread_pool: ThreadPool,

pub  thread_pool_clean: ThreadPool,
```

- 계정 로드/저장 작업의 병렬 처리
- 정리 작업 전용 스레드 풀

2. 메모리 매핑

- AppendVec 파일들을 메모리 매핑으로 접근
- 운영체제 레벨의 캐싱 활용
- 빠른 랜덤 액세스 제공

3. 지연 로딩

- 필요한 시점에만 계정 데이터 로드
- 메모리 사용량 최적화

오류 처리 및 복구

1. 참조 카운트 검증

```
fn exhaustively_verify_refcounts(&self, max_slot_inclusive: Option<Slot>) {
// 모든 계정의 참조 카운트 검증
self.accounts_index.get_and_then(entry.key(), |index_entry| {
if let Some(index_entry) = index_entry {
match (index_entry.ref_count() as usize).cmp(&entry.value().len()) {
// 참조 카운트 불일치 검출 및 처리
}
}
});
}
```

2. 데이터 일관성 보장

- 트랜잭션 실행 중 오류 발생 시 롤백
- 인덱스와 실제 데이터 간의 일관성 유지
- 정기적인 무결성 검사

요약

Accounts DB는 Solana의 고성능을 가능하게 하는 핵심 컴포넌트입니다:

- **효율적인 저장**: AppendVec 기반의 메모리 매핑된 파일 시스템
- **빠른 검색**: 다층 인덱스와 캐시 시스템
- **확장성**: 병렬 처리와 백그라운드 작업

- **최적화**: 정리, 압축, 지연 로딩 등의 최적화 기법
- **안정성**: 참조 카운트 검증과 데이터 일관성 보장

이러한 설계를 통해 Solana는 초당 수만 건의 트랜잭션을 처리하면서도 낮은 지연시간을 유지할 수 있습니다.